

Public Cloud Services IaC



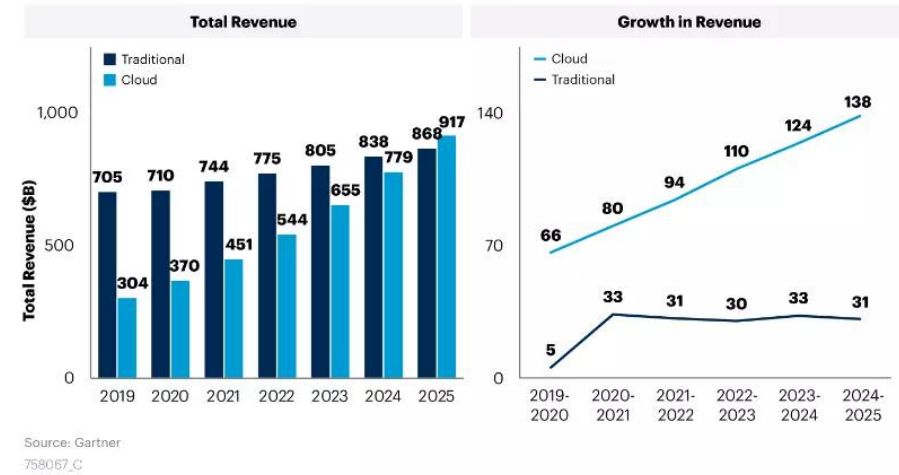
IaC in Practice



Software eats the world...



Figure 1: Sizing Cloud Shift, Worldwide, 2019 – 2025

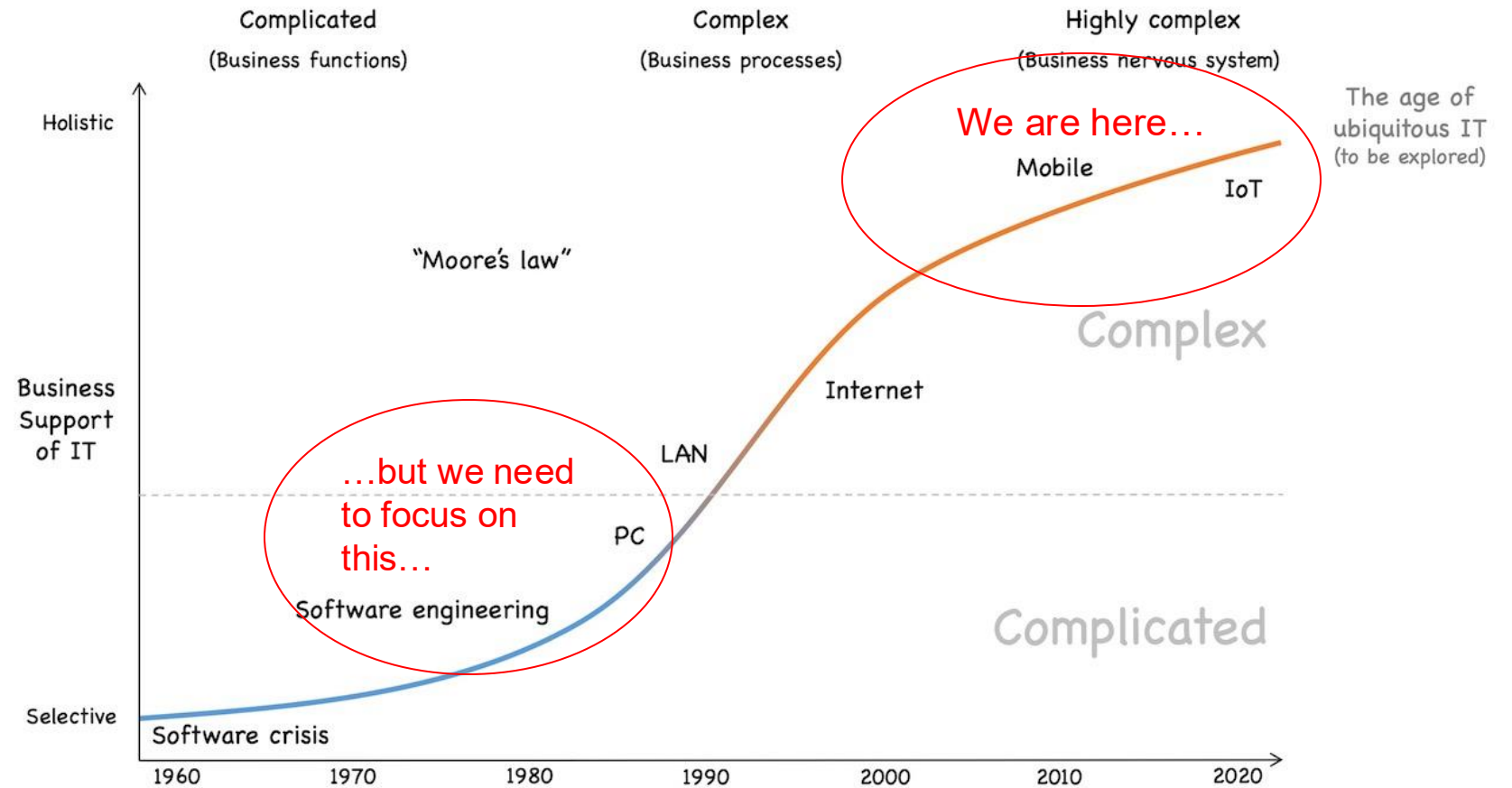


Gartner.

Workloads are shifting towards virtualized environments

→ Need for software engineering practices kicks in...

Problem Field



Modules for Softwaredevelopment	Modules for Softwareoperations
>20	<10
Programming Languages, Technologies, etc.	Processes, System Engineering, -management

Iron Age VS Cloud Age

Iron Age	Cloud Age
Physical hardware	Virtualized resources
Provisioning takes weeks	Provisioning takes minutes
Manual processes	Automated processes



Iron Age	Cloud Age
Cost of change is high	Cost of change is low
Changes represent failure (changes must be “managed,” “controlled”)	Changes represent learning and improvement
Reduce opportunities to fail	Maximize speed of improvement
Deliver in large batches, test at the end	Deliver small changes, test continuously
Long release cycles	Short release cycles
Monolithic architectures (fewer, larger moving parts)	Microservices architectures (more, smaller parts)
GUI-driven or physical configuration	Configuration as Code

5 Principles of Infrastructure in the Cloud Age

Assume that Systems are not reliable

- Complex Systems → multiple Failure Causes
- Upgrade/Update/Resize need reprovisioned Systems (or at least reboot)
- Systems need to be abstracted from underlying hardware

Make everything reproducible

- If you build a system, be sure you can rebuild it
- Stage-Parity: Test/Prod should be the same
- Enable horizontal scaling by supporting scale out

Create disposable systems

- Cloud infrastructure is as abstract as Software, it may be deleted as Software
- Infrastructure is not static but elastic. Add, alter and remove are natural operations

5 Principles of Infrastructure in the Cloud Age

Minimize Variations

- Each variation generates manual work or technical debt
- Operating 1000 services from the same kind is easier than operating 50 services of different kinds
 - Keep setup, OS, Architecture, Deployment Kind similar / the same

The process of creation / deployment should be common work

- Reproducible actions become robust, traceable, secure, tested, ...
- Bus-Faktor > 1
- Easy/Flexible Stage Parity
- Scripting, Scripting, Scripting

Pet VS Cattle



Pets

Legacy Infrastructure

Pets are given names like grumpycat.petstore.com They are unique, lovingly hand raised and cared for When they get ill, you nurse them back to health
Infrastructure is a permanent fixture in the data center
Infrastructure takes days to create, are serviced weekly, maintained for years, and requires migration projects to move
Infrastructure is modified during maintenance hours and generally requires special privileges such as root access
Infrastructure requires several different teams to coordinate and provision the full environment
Infrastructure is static, requiring excess capacity to be dormant for use during peak periods of demands
Infrastructure is an capital expenditure that charges a fix amount regardless of usage patterns

Cattle



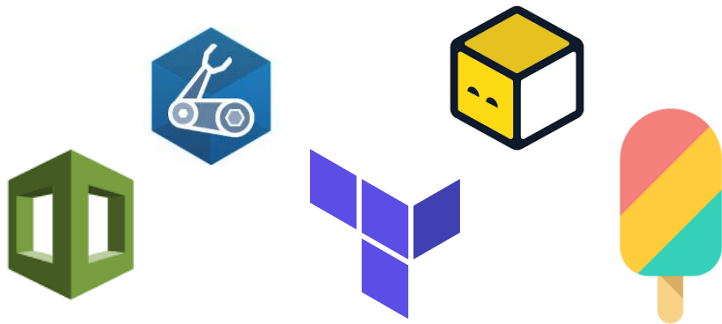
Cloud-Friendly Infrastructure

Cattle are given numbers like 10200713.cattlerancher.com They are almost identical to other cattle When they get ill, you replace them and get another
Infrastructure is stateless, ephemeral, and transient
Infrastructure is instantiated, modified, destroyed and recreated in minutes from scratch using automated scripts
Infrastructure uses version-controlled scripts to modify any service without requiring root access or privileged logins
Infrastructure is self-service with the ability to provision computing, network and storage services with a single click
Infrastructure is elastic and scales automatically, expanding and contracting on-demand to service peak usage periods
Infrastructure is a operating expenditure that charges only for services when they are consumed

Infrastructure as Code

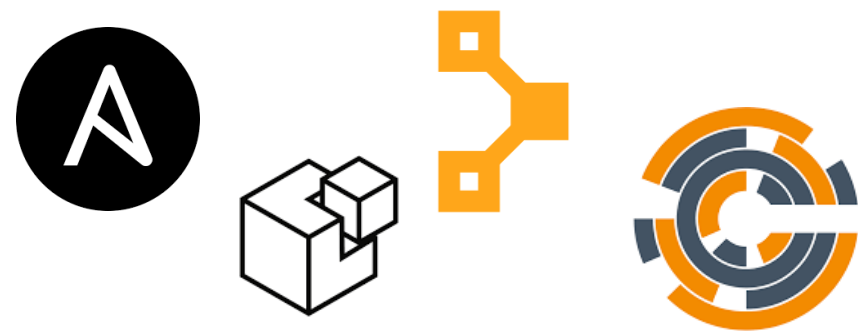
Infrastructure Provisioning

- Speaks with API of infrastructure provider
- Provisions "atomic" elements / services
- Seldomly adaption of existing services, re-creation instead
 - Be aware if you have data, data may be deleted of existing services
- Needs to handle state



Configuration Management

- Speaks with (IaaS)-servers / localhost
- Adapt instances and services
- Mostly adaption of existing services, re-creation only with remote services
 - Quite robust when it comes to existing services
- State is retrieved at runtime



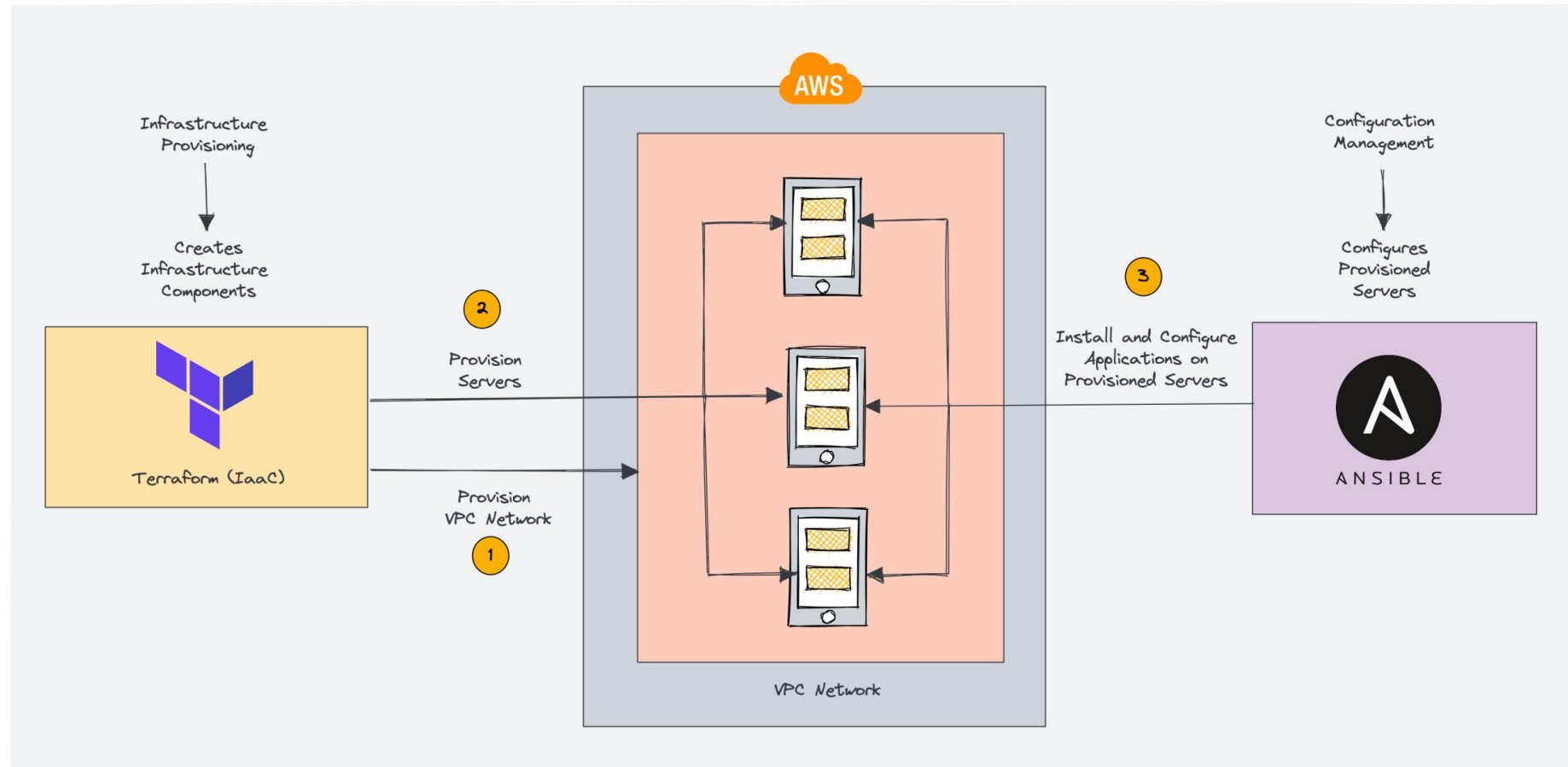
Survey...

Survey

Company Name	Tool Used	Main Use Case	Satisfaction Level (1-5)	Key Features Used	Challenges Faced
Company A	Ansible	Configuration Management	4	Playbooks, Roles	Complex Playbook Management
Company B	Terraform	Infrastructure Provisioning	5	HCL, State Management	Initial Setup Complexity
Company C	Both	Mixed (Infra & Configuration)	3	Ansible for Config, Terraform for Infra	Integration Issues, Learning Curve
Company D	Terraform	Cloud Infrastructure Automation	4	Modules, State Files	Handling Multi-Cloud Environments
Company E	Ansible	Application Deployment	4	YAML Playbooks, Roles	Scaling Issues
Company F	Terraform	Multi-Cloud Resource Management	5	Providers, Resource Plans	Terraform State File Management
Company G	Ansible	Orchestration	3	Ad-Hoc Commands, Roles	Complex Dependencies Management
Company H	Both	General Automation	4	Combination of Both Tools	Complexity in Managing Both Tools
Company I	Terraform	Infrastructure as Code	5	Modules, Workspaces	Initial Learning Curve
Company J	Ansible	System Configuration	4	Playbooks, Inventory Files	Handling Large Playbooks

<https://rjpn.org/jetnr/papers/JETNR2308001.pdf>

Infrastructure Provisioning ♡ Configuration Management



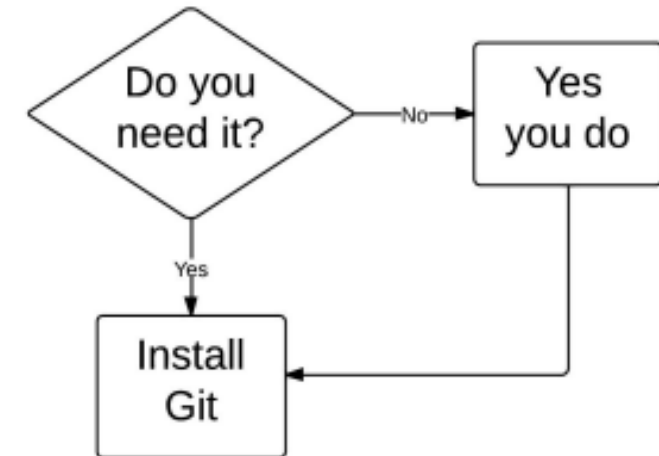
IaC is Source code

Sourcecode-Mgmt is a must have:

All configurations need to be under source control, even locally (e.g. etckeeper)

- Linting:
 - Syntaxchecks
 - Existieren für fast alle Sprachen
- Testing
 - Sematicchecks
- Auditing
 - Merge Request
- Versioning
 - Reverting

Version Control Flowchart



Pitfalls:

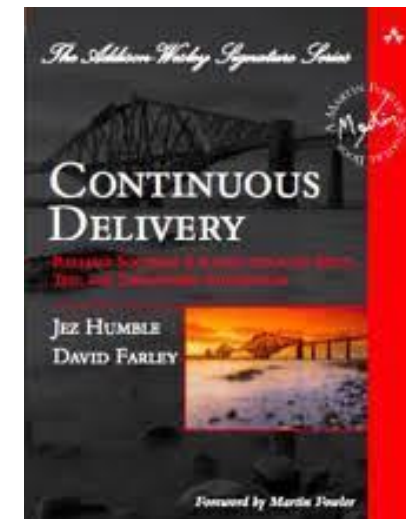
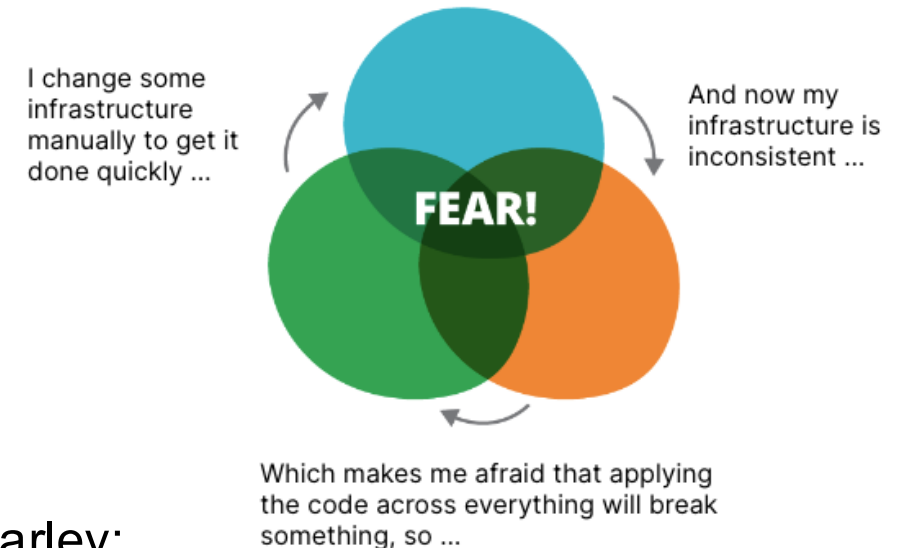
Fear the Fear Spiral

Infrastructure follows Code...including removal:

→ Lifecycle (including removal) is part of the game

Principles of Software Delivery from Jez Humble and David Farley:

1. *Create a Repeatable, Reliable Process for Releasing Software*
2. *Automate Almost Everything*
3. *Keep Everything in Version Control*
4. *If It Hurts, Do It More Frequently, and Bring the Pain Forward*
5. *Build Quality In*
6. *Done Means Released*
7. *Everybody Is Responsible for the Delivery Process*
8. *Continuous Improvement*

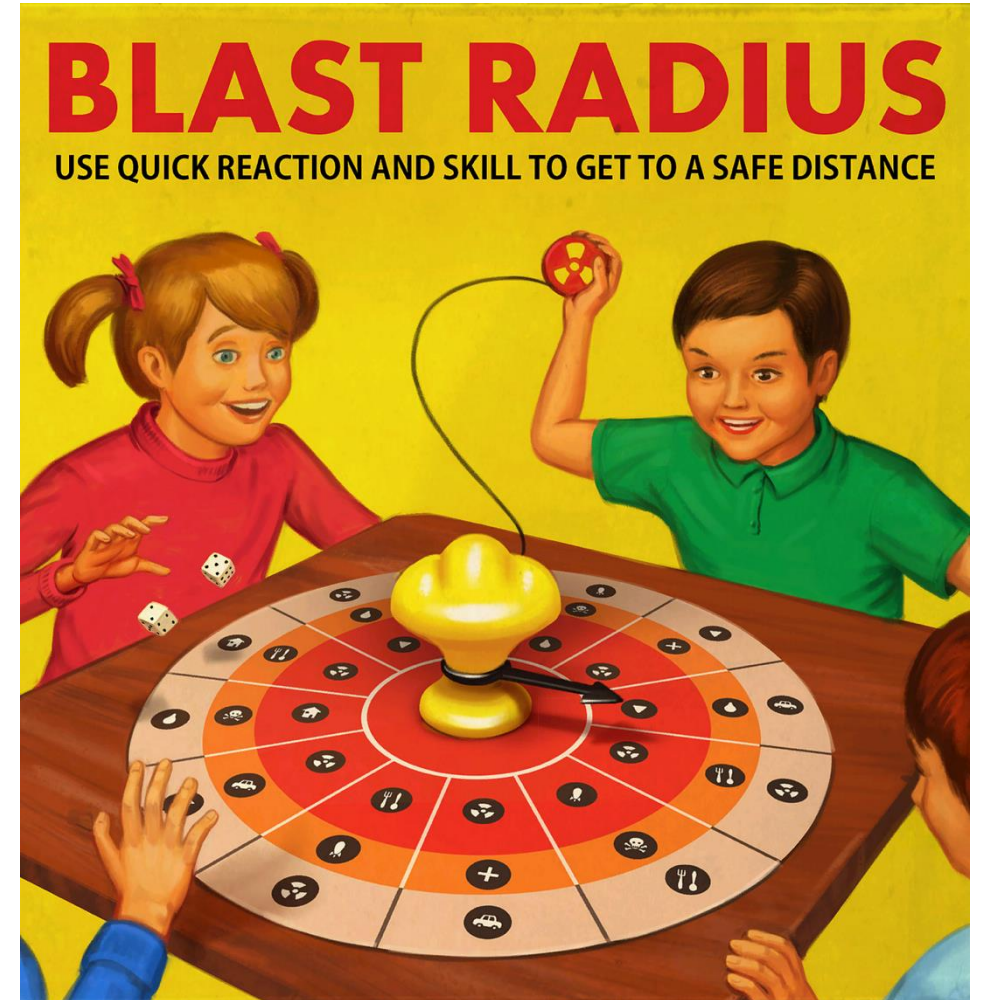


Pitfalls:

Mind the Blast Radius

IaC influences applications:

- We are on the bottom of the stack, if we are having failures here, all upper layers are affected
- Reducing side effects by defining fixed defined accounts / subscriptions / projects / namespaces / resource groups
 - Use testing / staging to become aware of blast radius
 - Use tagging / conventions of resources



Pitfalls:

How should infrastructure be defined in Sourcecode?

```
stack-tool create-virtual-server \  
  --name=my-server \  
  --os_image=ubuntu-22.10 \  
  --memory=4GB \  
  --disk=80GB
```

Pitfalls:

How should infrastructure be defined in Sourcecode?

```
stack-tool find-virtual-server --name=my-server
if [ "$?" = "1" ] ; then
    echo "Creating a new server"
    stack-tool create-virtual-server \
        --name=my-server \
        --os_image=ubuntu-22.10 \
        --memory=4GB \
        --disk=80GB
else
    echo "Server already exists, not creating a new one"
fi
```

Pitfalls:

How should infrastructure be defined in Sourcecode?

```
stack-tool find-virtual-server --name=my-server --print-id > saved-server-id
if [ "$?" = "1" ] ; then
    echo "Creating a new server"
    stack-tool create-virtual-server \
        --name=my-server \
        --os_image=ubuntu-22.10 \
        --memory=8GB \
        --disk=80GB
else
    echo "Server already exists, not creating a new one"
fi
CURRENT_RAM=`stack-tool get-virtual-server-info \
    --id=$(cat saved-server-id) \
    --attribute=memory`
if [ "${CURRENT_RAM}" != "8GB" ] ; then
    stack-tool modify-virtual-server \
        --id=$(cat saved-server-id) \
        --memory=8GB
fi
```


Pitfalls:

How should infrastructure be defined in Sourcecode?

```
virtual_machine:
  name: my_application_server
  source_image: 'base_linux'
  cpu: 2
  ram: 2GB
  network: private_network_segment
  provision:
    provisioner: servermaker
    role: tomcat_server
```

Imperative	Declarative
Explicite Instructions	Describe the outcome
Flexible	Domain Specific
High levels of control	Difficult to optimize
The system is stupid, you are smart	The system is smart, you don't care

Summary

- Declarative Languages to the rescue
 - Not caring how but what
 - Hard to debug
- Different tools for different levels
- Infrastructure Provisioning vs Configuration Management
 - Do not seek for the “one” solution
 - Try to combine best of worlds
 - Do not use different tools for same purpose



